Exam Overview

Duration: 4-5 days
Tech Stack: Node.js, Express.js, MongoDB (Mongoose)
Assessment Criteria: Code quality, API design, documentation

Project Requirements

Core Technologies (Required)

Table

Technology	Purpose
Node.js + Express	Server framework
MongoDB + Mongoose	Database & ODM
JWT	Authentication & authorization
bcrypt	Password encryption
Joi	Input validation
Nodemailer	Email verification
Multer	File uploads
Socket.io	Real-time notifications

Project Structure

```
├── config/                  # Env + config services
│   ├── .env
│   └── env.service.js
│
├── src/
│   ├── common/
│   │   └── email/
│   │       └── sendEmail.js
│   │
│   ├── middleware/         # Auth, multer, etc.
```

```
│   │   ├── auth.js
│   │   └── multer.js
│   │
│   ├── utils/
│   │   └── validation.js
│   │
│   ├── database/
│   │   ├── connection.js
│   │   └── model/
│   │       ├── message.model.js
│   │       └── user.model.js
│   │
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── auth.controller.js
│   │   │   ├── auth.service.js
│   │   │   └── auth.validation.js
│   │   │
│   │   ├── message/
│   │   │   └── (feature logic here)
│   │   │
│   │   └── users/
│   │       └── (feature logic here)
│   │
│   ├── app.controller.js
│   └── main.js
│
├── uploads/
├── node_modules/
├── .gitignore
├── package.json
└──  package-lock.json
```

Module 1: Authentication & User Management

1.1 Auth APIs (Required)

Table

Method	Endpoint	Description
POST	`/api/v1/auth/signup`	Register new user

Method	Endpoint	Description
POST	`/api/v1/auth/login`	Login user
POST	`/api/v1/auth/verify-email/:token`	Verify email
POST	`/api/v1/auth/resend-verification`	Resend verification email
POST	`/api/v1/auth/forgot-password`	Request password reset
POST	`/api/v1/auth/reset-password/:token`	Reset password

Signup Requirements:

- Validate email format, password strength (min 8 chars, 1 uppercase, 1 number)
- Hash password with bcrypt (salt rounds: 12)
- Send verification email via Nodemailer
- Generate JWT token upon email verification

Login Requirements:

- Return JWT access token (expires: 24h) and refresh token
- Include user role in token payload

1.2 User Profile APIs (Required)

Table

Method	Endpoint	Access	Description
GET	`/api/v1/users/profile`	User	Get own profile
PUT	`/api/v1/users/profile`	User	Update profile (name, phone, avatar)
DELETE	`/api/v1/users/profile`	User	Soft delete own account
POST	`/api/v1/users/upload-avatar`	User	Upload profile image

Soft Delete Logic:

- Add `isDeleted: Boolean` and `deletedAt: Date` fields
- Exclude soft-deleted users from all queries by default
- Add middleware to filter deleted records

Module 2: E-Commerce System

2.1 Category & Subcategory APIs

Dashboard (Admin Only):

Table

Method	Endpoint	Description
POST	`/api/v1/categories`	Create category
PUT	`/api/v1/categories/:id`	Update category
DELETE	`/api/v1/categories/:id`	Soft delete category
GET	`/api/v1/categories`	Get all categories (with subcategories)

Table

Method	Endpoint	Description
POST	`/api/v1/subcategories`	Create subcategory
PUT	`/api/v1/subcategories/:id`	Update subcategory
DELETE	`/api/v1/subcategories/:id`	Soft delete subcategory

Public APIs:

Table

Method	Endpoint	Description
GET	`/api/v1/categories`	List all active categories
GET	`/api/v1/categories/:id/subcategories`	Get subcategories by category
GET	`/api/v1/subcategories/:id`	Get subcategory details

2.2 Product APIs

Dashboard (Admin Only):

Table

Method	Endpoint	Description
POST	`/api/v1/admin/products`	Add product

Method	Endpoint	Description
PUT	`/api/v1/admin/products/:id`	Update product
DELETE	`/api/v1/admin/products/:id`	Soft delete product
PATCH	`/api/v1/admin/products/:id/stock`	Update stock quantity

Product Schema Requirements:

JavaScript

```
{
  name: String (required),
  description: String,
  price: Number (required, min: 0),
  stock: Number (required, min: 0),
  category: ObjectId (ref: Category),
  subcategory: ObjectId (ref: Subcategory),
  images: [String],
  isDeleted: Boolean,
  deletedAt: Date,
  autoDeletedAt: Date // When stock reached 0
}
```

Auto Soft-Delete Rule:

- When `stock` reaches 0, automatically set `isDeleted: true`
- Set `autoDeletedAt` timestamp for tracking

Public APIs:

Table

Method	Endpoint	Description
GET	`/api/v1/products`	Get all active products
GET	`/api/v1/products/:id`	Get single product details
GET	`/api/v1/products/category/:categoryId`	Filter by category
GET	`/api/v1/products/subcategory/:subcategoryId`	Filter by subcategory

Filtering & Pagination:

- Query params: `?page=1&limit=10&minPrice=10&maxPrice=100&sort=price_asc`

- Filter by price range within category/subcategory
- Sort by: price (asc/desc), createdAt, name

2.3 Cart APIs (Required)

Table

Method	Endpoint	Description
POST	`/api/v1/cart`	Add item to cart
GET	`/api/v1/cart`	View cart
PUT	`/api/v1/cart/:productId`	Update quantity
DELETE	`/api/v1/cart/:productId`	Remove item from cart
DELETE	`/api/v1/cart`	Clear entire cart

Add to Cart Logic:

- Check if product exists and is active
- Validate requested quantity ≤ available stock
- If item exists in cart, update quantity (ensure total ≤ stock)
- Calculate cart totals

2.4 Order APIs (Required)

Table

Method	Endpoint	Access	Description
POST	`/api/v1/orders/checkout`	User	Checkout cart
GET	`/api/v1/orders`	User	Get my orders
GET	`/api/v1/orders/:id`	User	Get order details
GET	`/api/v1/admin/orders`	Admin	Get all orders
PATCH	`/api/v1/admin/orders/:id/status`	Admin	Update order status

Checkout Requirements:

Payment Methods:

1. **Cash on Delivery (COD)** - Required implementation
   - Create order with status: `pending`

- Payment status: `pending`

2. **Visa/Card via Stripe** - !!!!Bonus implementation!!!!
   - Create Stripe payment intent
   - Handle payment confirmation webhook
   - Update order status on successful payment

Order Schema:

JavaScript

```
{
  user: ObjectId (ref: User),
  items: [{
    product: ObjectId,
    quantity: Number,
    price: Number // Snapshot price at purchase
  }],
  totalAmount: Number,
  paymentMethod: Enum ['cod', 'card'],
  paymentStatus: Enum ['pending', 'paid', 'failed'],
  orderStatus: Enum ['pending', 'processing', 'shipped', 'delivered',
'cancelled'],
  shippingAddress: Object,
  createdAt: Date
}
```

Stock Management:

- Deduct stock quantities on successful order creation
- If stock insufficient, return error with available quantity

---

Module 3: Real-Time Notifications (Socket.io)

Simple Admin Broadcast System

Requirements:

- Admin dashboard can send promotional messages/offers
- All connected users receive the message instantly
- Only authenticated admin can emit messages

Socket Events:

Table

Event	Direction	Description
`connection`	Server	Handle new connections
`authenticate`	Client → Server	Send JWT token
`admin:send-offer`	Client → Server	Admin broadcasts offer
`user:receive-offer`	Server → Client	Users receive offer
`disconnect`	Both	Handle disconnections

Implementation Flow:

1. User connects and authenticates with JWT
2. Admin emits offer with title, message, discount code
3. Server broadcasts to all authenticated users
4. Frontend displays toast/notification

Message Schema:

JavaScript

```javascript
{
  type: 'offer' | 'announcement',
  title: String,
  message: String,
  discountCode: String, // optional
  expiresAt: Date,
  createdAt: Date
}
```

Module 4: HR Management System

4.1 Staff Management (Admin Only)

Table

Method	Endpoint	Description
POST	`/api/v1/admin/staff`	Add staff member

Method	Endpoint	Description
GET	`/api/v1/admin/staff`	Get all staff
GET	`/api/v1/admin/staff/:id`	Get staff details
PUT	`/api/v1/admin/staff/:id`	Update staff info
DELETE	`/api/v1/admin/staff/:id`	Soft delete staff

Staff Schema:

JavaScript

```
{
  user: ObjectId (ref: User),
  dailySalary: Number (required), // Daily wage
  joinDate: Date,
  department: String,
  isActive: Boolean,
  monthlyReports: [{
    month: String, // "2024-03"
    totalDaysWorked: Number,
    totalDeductions: Number,
    finalSalary: Number,
    isPaid: Boolean,
    paidAt: Date
  }]
}
```

4.2 Attendance APIs

Table

Method	Endpoint	Description
POST	`/api/v1/staff/checkin`	Check in for the day
POST	`/api/v1/staff/checkout`	Check out for the day

Check-in Logic:

- Record check-in time
- If check-in after 9:00 AM, mark as late
- Prevent multiple check-ins per day

Check-out Logic:

- Record check-out time
- Calculate working hours
- If less than 8 hours, calculate deduction
- If no check-out by end of day, mark as absent

4.3 Salary & Deduction APIs (Admin Only)

Table

Method	Endpoint	Description
POST	`/api/v1/admin/staff/:id/deductions`	Add deduction
GET	`/api/v1/admin/staff/:id/deductions`	Get staff deductions
PUT	`/api/v1/admin/staff/:id/deductions/:deductionId`	Update deduction
DELETE	`/api/v1/admin/staff/:id/deductions/:deductionId`	Remove deduction

Deduction Schema:

JavaScript

```
{
  staff: ObjectId,
  month: String, // "2024-03"
  amount: Number,
  reason: String,
  date: Date
}
```

4.4 Monthly Salary API

Table

Method	Endpoint	Description
GET	`/api/v1/admin/staff/:id/salary/:month`	Calculate monthly salary
POST	`/api/v1/admin/staff/:id/salary/:month/pay`	Mark as paid
PUT	`/api/v1/admin/staff/:id/salary/:month/adjust`	Adjust salary (excuse)

Salary Calculation Logic:

plain

```
Base Salary = Daily Salary × Working Days in Month
Deductions = Late Days × (Daily Salary × 0.1) + Absent Days × Daily Salary +
Manual Deductions
Final Salary = Base Salary - Deductions + Adjustments
```

Mark as Paid:

- Set `isPaid: true`
- Record `paidAt` timestamp
- Generate salary slip (optional)

Bonus Points (Extra Credit)

Bonus 1: Testing with Cypress (+15 points)

- Write E2E tests for all auth flows
- Test cart operations
- Test checkout process
- Test admin CRUD operations

Required Tests:

JavaScript

```javascript
// Example test structure
describe('Authentication', () => {
  it('should signup new user', () => {});
  it('should verify email', () => {});
  it('should login user', () => {});
  it('should reset password', () => {});
});
```

Bonus 2: Cloud Storage Integration (+10 points)

- Upload images to **Cloudinary** or **ImageKit.io**
- Delete images when product/user is deleted
- Generate optimized image URLs

Bonus 3: Ticket Support System (+15 points)

Table

Method	Endpoint	Description
POST	`/api/v1/tickets`	Create support ticket
GET	`/api/v1/tickets`	Get my tickets
GET	`/api/v1/tickets/:id`	Get ticket details
POST	`/api/v1/tickets/:id/reply`	Add reply
PATCH	`/api/v1/admin/tickets/:id/status`	Update ticket status

Ticket Schema:

JavaScript

```javascript
{
  user: ObjectId,
  subject: String,
  description: String,
  status: Enum ['open', 'in-progress', 'resolved', 'closed'],
  priority: Enum ['low', 'medium', 'high'],
  replies: [{
    sender: ObjectId,
    message: String,
    isAdmin: Boolean,
    createdAt: Date
  }]
}
```

Database Schema Summary

Collections

1. **Users** - Authentication & profiles
2. **Categories** - Product categories
3. **Subcategories** - Category children
4. **Products** - E-commerce products
5. **Carts** - User shopping carts
6. **Orders** - Purchase orders
7. **Staff** - HR staff records

8. **Attendance** - Daily check-in/out
9. **Deductions** - Salary deductions
10. **Messages** - Socket.io broadcast history (optional)

Security Requirements

Required Implementations:

- [ ] JWT authentication middleware
- [ ] Role-based access control (RBAC)
- [ ] Password hashing with bcrypt
- [ ] Input sanitization (prevent NoSQL injection)
- [ ] Rate limiting on auth endpoints
- [ ] CORS configuration
- [ ] Helmet.js for security headers

Validation Requirements:

- All inputs validated with Joi
- File uploads restricted to images only (jpg, png, webp)
- Max file size: 5MB
- Validate MongoDB ObjectIds

Deliverables Checklist

Code (Required)

- [ ] Complete source code in GitHub repository
- [ ] `.env.example` file with all required variables
- [ ] `package.json` with all dependencies
- [ ] Proper folder structure

Documentation (Required)

- [ ] `README.md` with:
    - Project description
    - Installation instructions

- Environment variables list
- API endpoints documentation
- Socket.io events documentation

- [ ] Postman collection or API documentation

Database (Required)

- [ ] MongoDB connection setup
- [ ] All Mongoose schemas defined
- [ ] Proper indexes on frequently queried fields

Bonus Deliverables

- [ ] Cypress test suite
- [ ] Cloudinary/ImageKit integration
- [ ] Ticket system implementation

Grading Rubric (Total: 100 Points)

Table

Category	Points	Criteria
Auth & User	15	JWT, bcrypt, email verification working
E-Commerce Core	25	Products, cart, orders functional
Payment Integration	10	COD required, Stripe bonus
Categories & Filters	10	CRUD + filtering working
HR System	15	Attendance, salary, deductions
Socket.io	10	Real-time notifications
Code Quality	10	Clean code, error handling, validation
Documentation	5	README, API docs
Bonus Points	+40	Tests, Cloudinary, Tickets

Passing Score: 70/100
Excellent: 85+ with at least 1 bonus
Outstanding: 95+ with all bonuses

Getting Started Template

Environment Variables (.env)

env

```
# Server
PORT=5000
NODE_ENV=development

# Database
MONGODB_URI=mongodb://localhost:27017/ecommerce_exam

# JWT
JWT_SECRET=your_super_secret_key_here
JWT_EXPIRE=24h
JWT_REFRESH_SECRET=your_refresh_secret

# Email (Nodemailer)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_app_password

# Stripe (Bonus)
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...

# Cloudinary (Bonus)
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```

Package.json Dependencies

JSON

```
{
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^8.0.0",
    "bcryptjs": "^2.4.3",
    "jsonwebtoken": "^9.0.2",
    "joi": "^17.11.0",
    "nodemailer": "^6.9.7",
```

```
    "multer": "^1.4.5-lts.1",
    "socket.io": "^4.6.1",
    "dotenv": "^16.3.1",
    "cors": "^2.8.5",
    "helmet": "^7.1.0",
    "express-rate-limit": "^7.1.0",
    "stripe": "^14.0.0",
    "cloudinary": "^1.41.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.1", //if needed insted of the node --watch index.js
    "cypress": "^13.0.0",
    "supertest": "^10.2.0",
  }
}
```

Good luck
Build something you're proud of (ma3 eny 2a4ok XD)